Multi-Agent Loan-Default System

Project Progress Report — Internal Briefing for Presentation

Project team

2026–05–29

Abstract

We built a multi-agent loan-default scoring system for LendingClub Grade C–G loans. An XGBoost numeric baseline produces a per-borrower default probability; a panel of LLM agents reads the borrower's free-text description, audits a proposed text-fusion weight, and merges text-derived risk into the model in logit space. The system is exposed through a Streamlit single-page demo backed by a training notebook. The headline result is *not* that text fusion improves the global model — on a 4,199-row test set the Strategist AUC is essentially tied with baseline (+0.001, $p = 0.11$). The headline result is that the multi-agent system *learns to distrust text by default* and lets it act only on a high-confidence subset, where it does deliver a real +0.0142 AUPRC lift. The body of this document explains the architecture, data pipeline, and the evidence behind that finding so each team member can present a coherent slice.

1 System at a glance

There are two user-facing surfaces and one shared model artefact:

- `app/ui/Home.py` — single-page Streamlit demo. Loads the model pickle directly, calls the LLM endpoint inline. No backend service.
- `notebooks/loan_default.ipynb` — training notebook. Loads the consolidated text-analyst output, retrains XGBoost, runs the multi-agent strategy loop, writes `models/loan_default_model.pkl`.
- `evaluation/` — a 5-module offline toolkit (pipeline comparison, per-agent KPI, ablation, trust calibration, report). Driven by `notebooks/eval_verify_p3.py`, which writes `docs/eval_report_sample1.md` — the file the UI's "Offline evaluation" panel renders.

Five agents collaborate inside the loop:

Role	Type	Job
Text Analyst	LLM	Scores the borrower description into a risk score and self-reported confidence; falls back to a regex scorer when the LLM cannot be parsed.
Feature Strategist	LLM	Picks one of five `strategy_types` (`keep_learned`, `case_specific_exception`, `fuzzy_only`, `explanation_only`, `human_review`) and proposes a per-grade text weight. Strict rule: `case_specific_exception` must propose *strictly greater* than the learned weight.
Subgroup Advocate	LLM	Reasons about grade-level historical text impact; emits `pass / soft_veto / hard_veto`. Soft vetoes route to arbitration; hard vetoes are enforced.
Green Agent	LLM + ML	Hub: runs the XGBoost baseline, evaluates fusion, generates expectations and reflects (Reflexion), arbitrates on veto, and validates with adaptive thresholds. *The Arbitrator is Green — there is no separate White Agent.*
Reporter	LLM + det.	Writes a two-paragraph case audit: P1 covers the prediction result, P2 covers whether the text deserved to be trusted. Includes a Python-only deterministic fallback so the panel never goes blank.

2 Data pipeline

Text analysis is done offline by `text_analyst.py` (kept out of the repo because it carries user-specific API credentials). It runs against three sample csvs of 7,000 borrowers each and writes per-sample pickles to `text_ana_results/`. We consolidate those three runs into `text_ana_results/text_analysis_combined.pkl` (21,000 rows, 44 MB). Every consolidated row carries:

- the LendingClub loan `id` (recovered by joining each sample csv row-for-row, since within-file row order is verified to align with the pkl's `idx`)
- `_source_run`, `_source_idx`, `_global_idx` for full provenance
- all 30 original text-analyst fields (scores, confidences, raw LLM previews, fairness flags, etc.)

The training notebook `cell 12` now uses an `id`-join against this consolidated pkl. Replaying the stratified 80/20 split on `loan_default.csv` and inner-joining gives:

	Train	Test	Default rate
Old (sample1, idx-join)	∼250	60	0.250
New (combined, id-join)	**16,801**	**4,199**	0.222

That is roughly a 67× jump in training size and 70× in test size. All evaluation numbers below come from this new split.

3 The main finding

Key finding

Text fusion has near-zero benefit on the full test set, but a real positive lift on the high-confidence subset. The system has learned to distrust text by default and gate it through.

Evidence (combined.pkl test, $n = 4{,}199$):

- Per-grade ΔAUC at the learned weight is essentially zero for every grade: C −0.0005, D −0.0027, E +0.0001, F −0.0008, G −0.0030. Strategist vs baseline overall is ΔAUC = +0.0011, DeLong $p = 0.11$.
- Restrict to the top 35% of rows ranked by text confidence and the gap reverses: fused AUPRC beats baseline AUPRC by +0.0142. Confidence-vs-accuracy slope is +1.40 (high claimed confidence really is more reliable).
- The learned `best_strategy` reflects exactly that: *small weights everywhere, strict gate.*

The learned policy, side by side with the previous (small-sample) model:

	C	D	E	F	G	`conf_threshold`
Old (sample1, $n \approx 250$)	0.03	**0.13**	0.08	0.06	0.03	0.67
New (combined, $n = 16{,}801$)	0.03	**0.10**	0.05	0.03	0.03	**0.65**

In sample1, the D weight of 0.13 looked like genuine text benefit. At 67× the data, that benefit shrinks to a 0.10 weight whose own `grade_train_stats` entry is mildly negative (−0.003). The same relaxation happens on E and F. The takeaway for a presentation slide:

> *"The small-sample model was over-confident about how useful text is. At real scale the multi-agent system converges to a much more conservative policy: it touches text only on the cases where the borrower description is concrete enough to earn its place, and it pays for itself there."*

4 Evaluation numbers worth quoting

Pipeline head-to-head (test set, $n = 4{,}199$):

Pipeline	AUC	AUPRC	DeLong p vs baseline
baseline (XGBoost only)	0.6138	0.3001	—
MAS Strategist (fused)	0.6149	0.3015	0.11

Ablation — is the multi-agent system worth its complexity?

Method	Test AUC	Δ vs baseline	Note
Baseline only	0.6138	0.0000	XGBoost on 16 numeric features
LR stacking	0.6094	−0.0045	LogReg over numeric + text + regex flags
Grid search (best)	0.6142	+0.0003	1,620-combo brute-force grade-weight grid
MAS Strategist	**0.6149**	**+0.0011**	Multi-agent LLM-tuned 5-scalar weights

Two readings of the same table:

- Pessimist: MAS only beats brute-force grid search by +0.0008 AUC.
- Honest: with this baseline strength and this much data, the Strategist matches an exhaustive brute-force search while making five small *interpretable* decisions per grade. The win is *auditability + selective use of text*, not raw AUC.

Trust calibration (the headline section). The offline report's `trust_calibration` verdict on combined.pkl reads:

> *"confidence is a real trust signal; selective fusion helps"*

This is the affirmative branch of a four-way verdict tree. The previous sample1 report instead read "*confidence tracks accuracy but fusion gains are flat at the top*" — the noisy small-sample reading. The 67× data jump turned an inconclusive picture into the one the system was designed for.

5 Implementation notes worth defending

Three engineering conventions are load-bearing and should be preserved when extending the system. They are documented in the README and recapped here so a team member can answer a "why is it like this" question:

1. **All LLM calls use JSON output mode.** MiniMax-M2.7 is a thinking model. In free-text mode it leaks `<think>...</think>` blocks or stream-of-consciousness reasoning that exhausts the token budget *before* producing the actual answer — the user-visible output ends up blank or truncated. JSON mode reliably suppresses this. Every agent (Text Analyst, Strategist, Advocate, Green LLM, Arbitrator, Reporter) emits a JSON object that the UI parses locally with `_extract_json_object`.
2. **The Reporter has a deterministic Python fallback.** `_reporter_deterministic_fallback` synthesises the two-paragraph narrative from numeric inputs alone, matching the prompt's structure (P1: prediction result, P2: trust assessment; with separate branches for `effective_weight = 0` vs > 0). The Reporter panel can never go blank, even when the LLM returns invalid or truncated JSON.
3. **The `reasoning` field carries borrower-level analysis, never system status.** `fallback_text_analyst` writes `reasoning` as a sentence about the borrower's text; system-level failure detail (e.g. "LLM JSON parsing failed: ...") goes into a separate `_fallback_reason` field that never flows into another agent's prompt. This is what stops the Reporter from confusing system state for borrower narrative quality.

6 Repository status and how to run

Current branch: `main` on https://github.com/junwei-li057/loan-mas-project. Most recent commit at the time of writing is the consolidated retrain (`058855f`). The pipeline is self-consistent: model trained on combined.pkl, offline report regenerated on the same split.

To run the demo (no LLM credits required):

```
pip install -r requirements.txt
streamlit run app/loan_demo.py
```

The XGBoost baseline, four preset borrowers, and "Offline evaluation" panel work without an API key. Setting `MINIMAX_API_KEY` activates the LLM agent panels.

To regenerate the report after retraining:

```
python notebooks/eval_verify_p3.py
```

Suggested presentation flow (15–20 min):

1. Motivation: numeric features alone leave free-text on the table; but naive text fusion is risky.
2. Architecture: walk through the 5 agents using the table in Section 1; emphasise that Green plays both ML hub and arbitrator.
3. Live demo of **Case 1 (Grade C, hard veto)** or **Case 2 (Grade E, arbitration)** — shows the full Strategist → Advocate → Green flow on a single borrower.
4. The main finding (Section 3), framed as "*the system learned to distrust text by default*". Show the weight table to make the small-sample-overconfidence story concrete.
5. Ablation table (Section 4) as a credibility anchor: MAS matches an exhaustive brute-force grid search with five interpretable scalars.
6. Close on the trust calibration verdict: the design assumption was vindicated only at scale.